MIT 8103 Advanced Database Systems

Ride-hailing / Logistics System · Theresa Amaechi · Student ID 301918155

Case study

For my case study, I chose a Ride-Hailing / Logistics System since I currently work with a Logistics firm. I designed a database that helps dispatchers manage trip requests and assign drivers to riders. It also allows operations analysts to review trip activities and keep track of the revenue generated by drivers. The database has four main tables: Rider, Driver, Trip, and Payment. Each table stores information relevant to its purpose and helps the system manage records properly. The main operations supported by the database include registering riders and drivers, requesting trips, assigning drivers, updating trip statuses and recording payments for completed trips. my aim is to make trip management easier, improve coordination between riders and drivers, and help the operations team keep accurate records of trips and payments.

Portfolio 1: Database Design and Modelling

I designed the relational database using four tables, with each table serving a specific purpose. I used primary keys to uniquely identify records and foreign keys to connect related information across the tables. I also added check constraints to make sure the data entered follows the required rules. One of the rules I implemented ensures that a driver cannot be assigned to more than one active trip at a time. This is done using a partial unique constraint on `driver_id` for trips with a status of `assigned` or `in_progress`. I also demonstrated how the sample data meets the requirements of 1NF, 2NF and 3NF. To test the database, I ran 34 intentionally incorrect insert statements. All were rejected, and the valid sample records remained unchanged.

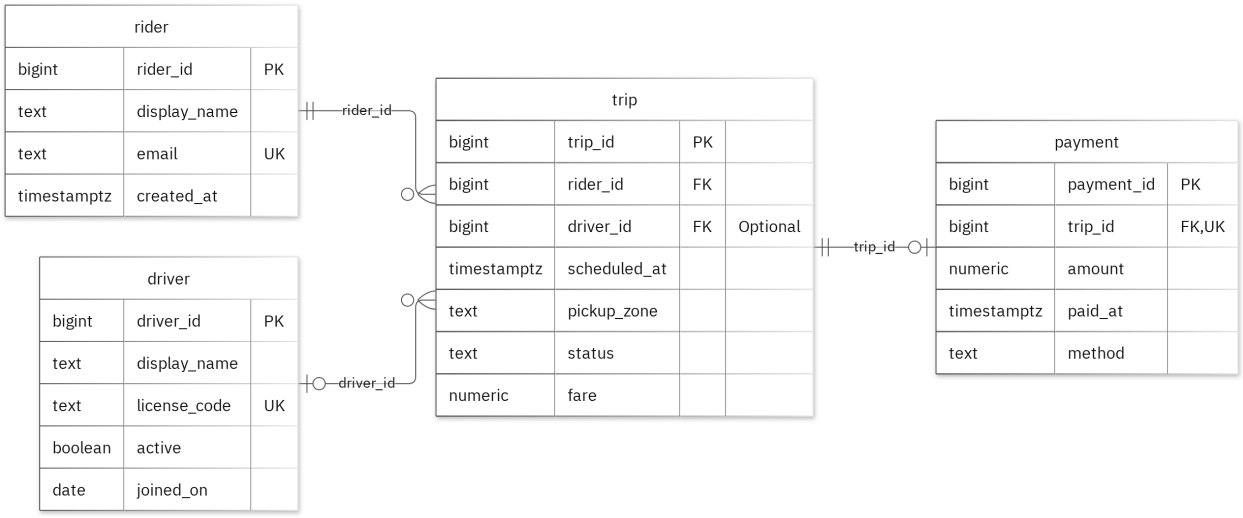


Figure 1: Ride-hailing / Logistics System ER diagram

Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I tested the sample records using four SQL queries. The join query returned two results, including Blake Rider paired with Casey Driver. For the index performance test, I created 20,000 separate performance records and compared the lookup times before and after adding the index. The median time dropped from 1.071 ms to 0.234 ms. Both tests returned the same records, and the execution plan confirmed that the new index was being used. These results show that the index improved the lookup speed in my test, although the timings are specific to the query and environment I used.

Portfolio 3: Transactions and Concurrency

I utilised a temporary counter on trip to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used a MongoDB collection called `trip_events` to store the event records for my case study. I added two sample events and tested them with four queries, which returned the expected results. I also tested the validation rules, and the system rejected an invalid date format and a duplicate event ID. I used embedded snapshots to keep related information together within each event. The `details` field can also hold different information depending on the type of event being recorded. The events used for this test are fictional, while PostgreSQL remains responsible for enforcing the main relational database rules.

Repository and evidence

https://github.com/tamaechi8155-rgb/Amaechi_Theresa_301918155

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed using three MongoDB replica members in separate failure zones to improve availability. I explained how primary elections, majority acknowledgement, secondary lag and sharding would work for `trip_events`. Replication helps keep the system available, while backups support data recovery. PostgreSQL also needs a separate availability plan. This was only a proposed design; I did not deploy or test a live replica set or failover.

Individual reflection

I placed the rider, driver, trip and payment in separate tables to avoid repeating information. I added a rule to ensure that a driver can only have one active trip at a time. I also tested the constraints to confirm that the required values and table relationships were maintained. The two-session test showed that both sessions could read the same value before updating it. Locking the row before reading helps prevent this issue and ensures both updates are saved. The index test also showed me why checking the execution plan is important, because two queries can return the same result but require different amounts of database work ## AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams